

Greedy Method and Huffman Encoding

Jianguo Lu

March 31, 2025

The Greedy Method

- ▶ The greedy method is a general algorithm design paradigm, built on the following elements:
 - ▶ Configurations: different choices, collections, or values to find
 - ▶ Objective function: a score assigned to configurations, which we want to either maximize or minimize
- ▶ It works best when applied to problems with the greedy-choice property:
 - ▶ A globally-optimal solution can always be found by a series of local improvements from a starting configuration.

An example of greedy method: Text Compression Problem

- ▶ Given a string X , efficiently encode X into a smaller string Y
- ▶ Saves memory and/or bandwidth
- ▶ A good approach: Huffman encoding
 - ▶ Compute frequency $f(c)$ for each character c .
 - ▶ Encode high-frequency characters with short code words
 - ▶ No code word is a prefix for another code
 - ▶ Use an optimal encoding tree to determine the code words
- ▶ Used widely in text compression, such as ZIP.

ASCII encoding

char	ASCII	binary
h	104	01101000
a	97	01100001
p	112	01110000
y	121	01111001
i	105	01101001
o	111	01101111
space	32	00100000

- ▶ The string "happy hip hop" would be encoded in ASCII as

104 97 112 112 121 32 104 105 112 32 104 111 112

- ▶ Stream of bits is:

01101000 01100001 01110000 01110000 01111001 00100000
01101000 01101001 01110000 00100000 01101000 01101111
01110000

- ▶ $13 * 8 = 104$ bits needed

A more compact encoding

char	number	binary
h	0	000
a	1	001
p	2	010
y	3	011
i	4	100
o	5	101
space	6	110

- ▶ The string "happy hip hop" needs $39 = 13 * 3$ bits
- ▶ Problem: the mapping is not standard. Need to attach a mapping table.
- ▶ How to reduce the size further?
- ▶ Note that 'h' is used more often than 'o'. Use shorter encoding for more frequent letters.

A Variable-Length Encoding

char	bit pattern
h	01
a	000
p	10
y	1111
i	001
o	1110
space	110

01	000	10	10	1111	110	01	001	10	110	01	1110	10
----	-----	----	----	------	-----	----	-----	----	-----	----	------	----

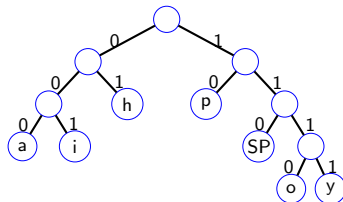
- ▶ It uses less bits (34) for the same string.
- ▶ How to translate the following encoding back to the string?
0100010101111110010011011001111010
- ▶ Note that there are no encoding boundaries
 - ▶ In fixed length encoding, we can utilize the length (e.g., every three bits)
 - ▶ In variable length encoding, how can we separate those codings?

Prefix encoding

- ▶ Prefix property: no character's encoding is a prefix of any other
 - ▶ e.g. if 'h' is encoded with 01 then no other character's encoding will start with 01.
 - ▶ It also implied that no character is encoded to just 0.
 - ▶ Better to be called 'no prefix' encoding.
- ▶ There is no ambiguity in determining where the character boundaries are.
- ▶ Simply find matches starting from the beginning

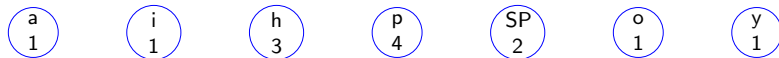
Encoding Seen as a Tree (Huffman Tree)

- ▶ An encoding tree represents a prefix code
- ▶ Each left-going edge represents a 0, each right-going edge a 1.
- ▶ Each external node stores a character
- ▶ The code word of a character is given by the path from the root to the external node storing the character (0 for a left child and 1 for a right child)
 - ▶ e.g., the path to 'h' is '01'
 - ▶ ex: How to decode **111100110101111** ?
- ▶ How to generate such a tree?



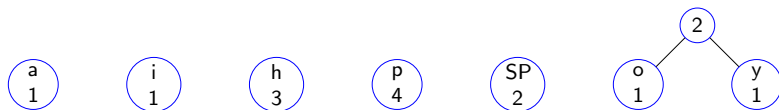
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



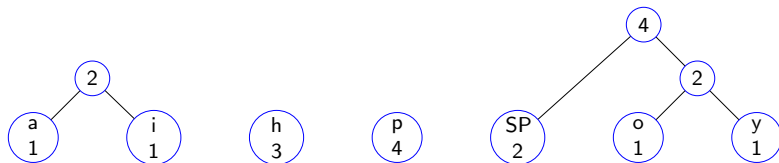
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



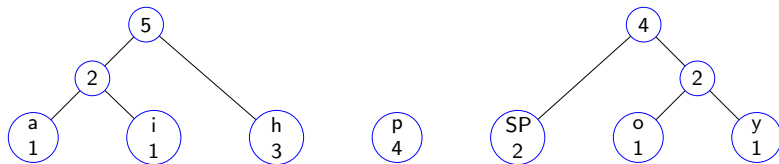
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



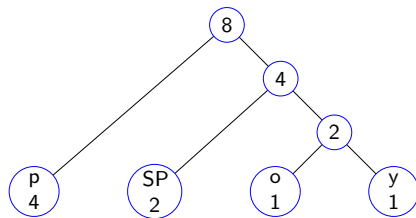
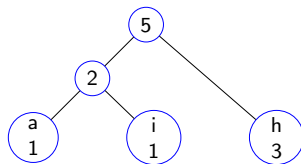
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



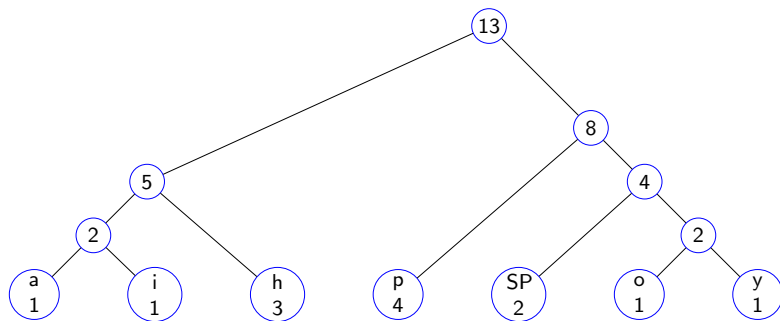
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



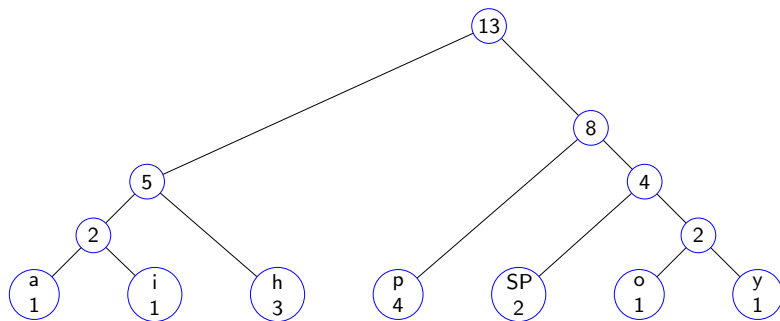
Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



Tracing the Algorithm

1. Start with single nodes with their frequencies as the weights
2. Pick two trees with the smallest weights, combine them into a new tree.
The weight of the root is equal to the sum of the left and right children.
3. Repeat Step 2 until only one tree is left.



Huffman's Algorithm

Algorithm 1: HuffmanEncoding(X)

Input : string X of size n

Output: optimal encoding trie for X

```
1  $C = \text{distinctCharacters}(X)$ 
2  $\text{computeFrequencies}(C, X)$ 
3  $Q = \text{empty heap}$ 
4 for all  $c \in C$  do
5    $T = \text{new single-node tree storing } c$ 
6    $Q.\text{insert}(\text{getFrequency}(c), T)$ 
7 end
8 while  $Q.\text{size}() > 1$  do
9    $f1 = Q.\text{minKey}()$ 
10   $T1 = Q.\text{removeMin}()$ 
11   $f2 = Q.\text{minKey}()$ 
12   $T2 = Q.\text{removeMin}()$ 
13   $T = \text{join}(T1, T2)$ 
14   $Q.\text{insert}(f1 + f2, T)$ 
15 end
16 return  $Q.\text{removeMin}()$ 
```


Complexity of Huffman's Algorithm

- ▶ Given a string X , Huffman's algorithm constructs a prefix code that minimizes the size of the encoding of X
- ▶ It runs in time $O(n + d \log d)$,
 - ▶ n : size of X
 - ▶ d : the number of distinct characters of X
 - ▶ In each iteration, it takes two min values. When implemented using priority queue, the time complexity is $O(\log d)$.
 - ▶ There are $d - 1$ iterations. Hence $O(d \log d)$
 - ▶ Frequency count is $O(n)$
- ▶ A heap-based priority queue is used as an auxiliary structure.

- ▶ Bonus assignment [here](#) (clickable)

```
public static void main(String[] args) throws Exception {  
    // build the Huffman tree  
    Node tree = buildTree(args[0]);  
    //construct the mapping table  
    String[] codeTable = new String[255];  
    getTable(codeTable, tree, "");  
  
    //compress the string  
    String code = compress(args[0], codeTable);  
    System.out.println(code);  
  
    //decode the string  
    String decode = expand(code, tree);  
    System.out.println(decode);  
}  
}
```

Java code

```
public class A6 {

    private static final int CHAR_RANGE = 256;

    //Compressing: look up the codeTable for each char in the input
    public static String compress(String s, String[] codeTable) {
        StringBuffer out = new StringBuffer();
        for (int i = 0; i < s.length(); i++) {
            out.append(codeTable[s.charAt(i)]);
        }
        return new String(out);
    }

    // get the mapping table from the Huffman Tree for the encoding for
    // all characters.
    // e.g. 'a'—> 001
    static void getTable(String[] code_table, Node node, String s) {
        if (!node.isLeaf()) {
            getTable(code_table, node.left, s + '0');
            getTable(code_table, node.right, s + '1');
        } else {
            code_table[node.ch] = s;
        }
    }
}
```

Java code

```
public static String expand(String code, Node tree) {
    Node x = tree;
    StringBuffer s = new StringBuffer();
    int i=0;
    while (i < code.length()) {
        while (!x.isLeaf()) {
            if (code.charAt(i) == '0') {
                if (x.left != null)
                    x = x.left;
            }
            else {
                if (x.right != null)
                    x = x.right;
            }
            i++;
        }
        s.append(x.ch);
        x = tree;
    }
    return new String(s);
}
```

Java code

```
private static Node buildTree(String s) {
    StringBuffer out = new StringBuffer();
    int[] freq = new int[CHAR_RANGE];
    for (int i = 0; i < s.length(); i++)
        freq[s.charAt(i)]++;

    PriorityQueue<Node> pq = new PriorityQueue<Node>();
    for (char c = 0; c < CHAR_RANGE; c++)
        if (freq[c] > 0)
            pq.add(new Node(c, freq[c], null,
                            null));

    while (pq.size() > 1) {
        // add the missing lines here
        // You should remove two minimum nodes from
        // the priority queue
        // create a parent node for those two nodes
        // add that parent node into the queue
    }

    return pq.remove();
}
```

Takeaways

- ▶ Huffman encoding. variable length prefix encoding.
- ▶ Huffman tree, Huffman algorithm. Greedy method. Complexity of the algorithm.
- ▶ Readings: p 595-597 in the book.